Graph Review Sheet

A graph is Tree generalized: drop the "must have a root, no cycles" restriction, and nodes can connect however they like. The DFS/BFS you learned for trees carry over almost verbatim here — with exactly one addition: **a visited set to stop cycles from looping forever**. This book walks from the concept to topological sort, with diagrams, and covers the questions you asked along the way.

Python 3 · Tutor style · Includes diagrams · Includes FAQ · Includes Grid/DAG

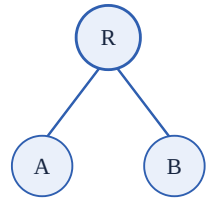
00 Review Order

Start with the concept plus the two traversals (DFS/BFS — old friends already), then layer on applications one at a time: can I reach it → how many pieces → fewest steps → 2D grids → directed → priority ordering.

> **One sentence covering the whole book:** nearly every graph problem is, at its core, "DFS or BFS + a visited set + a little extra tracking (distance / size / direction)." Burn that sentence into memory — from here on, for every problem you'll be able to recognize "which traversal is this, and what extra thing am I tracking."

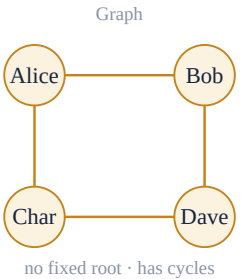
01 Graph Fundamentals

Tree is a **special case** of graph: it has a root, parent-child levels, and **no cycles**. Graph is more free-form: no root required, nodes can connect however they like, **cycles are allowed**, and it can be undirected or directed.



has root · no cycles · levels

Tree = a special Graph



no fixed root · has cycles

Four people who all know each other, a closed loop

02 Adjacency List

In Python, a graph is most commonly represented with a `dict`: keys are nodes, values are the list of their neighbors. This is the input format nearly every graph function in this book expects.

An adjacency list looks like this

graph

```
graph = {
    "Alice":   ["Bob", "Charlie"],
    "Bob":     ["Alice", "David"],
    "Charlie": ["Alice", "David"],
    "David":   ["Bob", "Charlie"],
}
# read it as: graph["Alice"] is the complete list of Alice's neighbors
```

Compare with Tree: a TreeNode has two named pointers, `left` / `right`. Graph uses **one neighbor list** instead, because a node can have arbitrarily many neighbors — it's no longer limited to just two directions.

03 What the visited set is for

A tree has no cycles, so traversal naturally terminates. A graph **can** have cycles (Alice→Bob→Alice→...), and without a guard, that's an infinite loop. `visited` is exactly that guard.

no visited → Alice⇄Bob loops forever

A cycle = walking back to a node you've already visited

Q Once a node is in visited, does that mean it's "ignored" and not counted?

No. The **first** time a node is visited, it's already been **fully counted** (added to result, counted toward size, or counted into a component). Skipping it on later encounters is to **avoid double-counting**, not to treat it as if it doesn't exist. Example: in `largest_component`, if A connects to B, and B connects to C/D/E, then one DFS starting from A has already counted B, C, D, and E into that component's size. If the outer loop later reaches B and skips it, that's purely because B was already counted during A's DFS — skipping it prevents counting the same component twice.

04 DFS — Depth-First Search

Go one path all the way to the end, then backtrack. Identical to Tree's DFS, just with `visited` added to guard against cycles.

dfs — recursion + visited

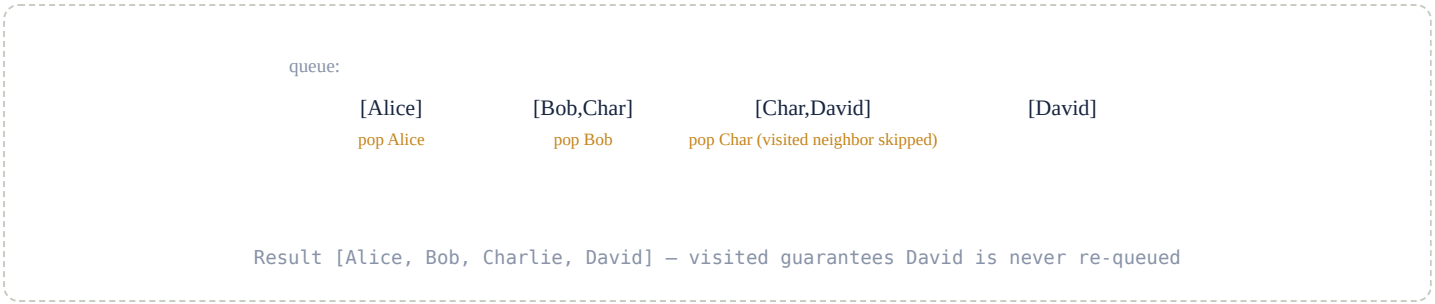
dfs

```
def dfs(graph, start):
    visited = set()
    result = []

    def helper(node):
        visited.add(node)              # mark first, then process — prevents re-entry
        result.append(node)
        for neighbor in graph[node]:
            if neighbor not in visited:  # only visit neighbors not yet visited
                helper(neighbor)

    helper(start)
    return result
```

The one and only difference from Tree's DFS: Tree's recursive call names two specific children, `left` / `right`; Graph's recursive call loops `for neighbor in graph[node]`, since the number of neighbors isn't fixed — and it **must** check `visited` before recursing, or a cycle will loop it to death forever. That `if` is the one extra line Graph's DFS has over Tree's.

05 BFS — Breadth-First Search

Sweep level by level with a queue. Structurally identical to Tree's `level_order`, with the same `visited` added.



bfs — queue + visited bfs

```
def bfs(graph, start):
    visited = set()
    queue = [start]
    result = []
    visited.add(start)                # mark right when it's queued, NOT when it's popped!

    while len(queue) > 0:
        node = queue.pop(0)
        result.append(node)
        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)
    return result
```

Q Why does visited.add happen at enqueue time, not dequeue time?

If you only mark on dequeue, the same node could be pushed into the queue again by a **different** edge while it's still waiting in line — resulting in duplicate entries and the same node being visited twice. Marking **the moment it's enqueued** guarantees every node enters the queue exactly once.

	USES	TRAVERSAL STYLE
DFS	Recursion / stack	One path all the way down, then backtrack
BFS	queue	Level by level, nearest to farthest

06 has_path — Can I Reach It?

Only asks True/False — no need to accumulate a result. **Return True the instant you find it** — that's the biggest difference from `dfs()` / `bfs()` collecting the entire result.

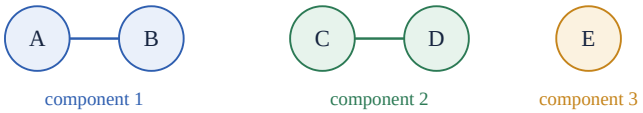
has_path — DFS version, no result needed path

```
def has_path(graph, start, target):
    if start not in graph:
        return False
    visited = set()

    def helper(node):
        if node == target:            # found it, return True right away, no need to finish traversing
            return True
        visited.add(node)
        for neighbor in graph[node]:
            if neighbor not in visited:
                if helper(neighbor):  # the moment a sub-call returns True, propagate it upward
                    return True
        return False                  # tried every neighbor, not found

    return helper(start)
```

This is the same pattern as Tree's search_tree: when asking "does it exist," return True early and let it propagate up layer by layer — you don't need to collect a result list. Recall the mantra from Book 02: "how many → use +; does it exist → use or / early return."

07 count_components — Connected Components

A graph might not be one single piece — some nodes may have no path connecting them at all. Every time you discover an unvisited node, you've found a brand-new isolated region.



A-B, C-D, E (isolated) → 3 connected components total

count_components — outer loop + inner DFS components

```
def count_components(graph):
    visited = set()
    count = 0

    def visit(node):
        visited.add(node)
        for neighbor in graph[node]:
            if neighbor not in visited:
                visit(neighbor)

    for node in graph:                # outer: sweep every node in the graph
        if node not in visited:       # unvisited → discovered a new piece
            count += 1
            visit(node)                # mark the entire piece
    return count
```

Structural memory hook: "outer for + inner DFS" is the shared skeleton behind `largest_component` and `count_islands` later in this book — the outer loop is responsible for "discovering a new region," the inner one for "fully exploring that region." Learn this one structure, and the next three functions are almost the same template copy-pasted.

08 shortest_path — Shortest Path

Why BFS and not DFS? Because BFS expands **level by level** — the first time it reaches target, the number of steps taken is necessarily the minimum.

shortest_path — queue carries a distance along with it bfs+dist

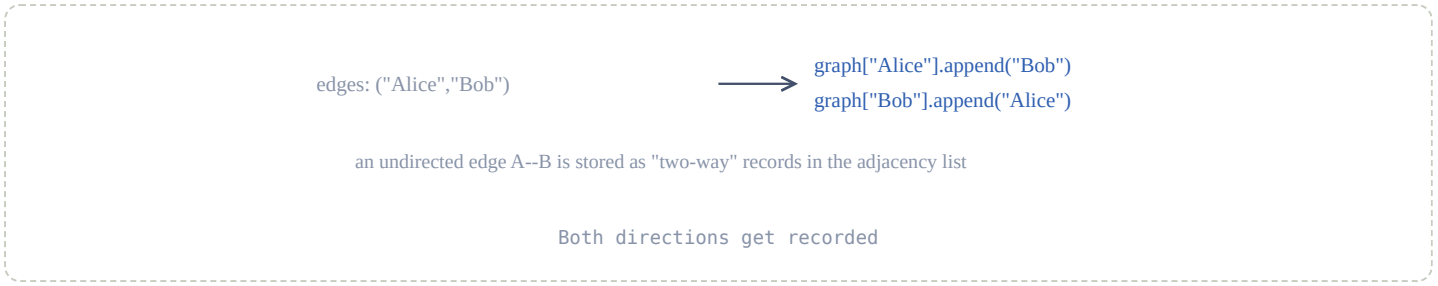
```
def shortest_path(graph, start, target):
    if start not in graph or target not in graph:
        return -1
    visited = set()
    visited.add(start)
    queue = [(start, 0)]               # tuple: (node, how many steps to get here)

    while len(queue) > 0:
        node, distance = queue.pop(0)
        if node == target:
            return distance             # first time reaching target IS the shortest
        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append((neighbor, distance + 1))
    return -1
```

Q Why can't DFS guarantee the shortest path?

DFS will charge headfirst down some path all the way to the end, even if that particular path takes the scenic route. **The first path it finds is not necessarily the shortest one.** BFS expands in the order "distance 0, distance 1, distance 2, …," which guarantees that the number of steps used the first time it touches target is the minimum. That's also exactly why `shortest_path` 's queue carries an extra `distance` alongside the node.

09 build_graph — Building a Graph

Convert an edge list (individual connections) into an adjacency list. For an undirected graph: **one edge gets added on both sides**.



build_graph — undirected, add both sides build

```
def build_graph(edges):
    graph = {}
    for node_a, node_b in edges:
        if node_a not in graph:
            graph[node_a] = []
        if node_b not in graph:
            graph[node_b] = []
        graph[node_a].append(node_b)   # A can reach B
        graph[node_b].append(node_a)   # undirected → B can reach A too
    return graph
```

10 largest_component — Largest Connected Component

Same skeleton as count_components — the inner DFS is just upgraded from "marking" to "also counting the size while it's at it."

largest_component — DFS returns subtree size size

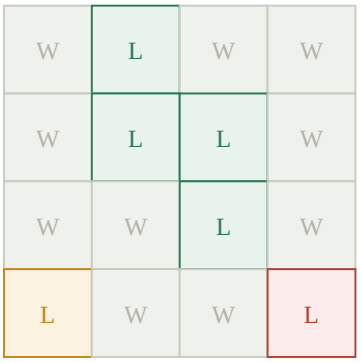
```
def largest_component(graph):
    visited = set()
    largest = 0

    def explore_size(node):
        visited.add(node)
        size = 1                       # count myself as 1 first
        for neighbor in graph[node]:
            if neighbor not in visited:
                size += explore_size(neighbor)   # add on everything reachable through the neighbor
        return size

    for node in graph:
        if node not in visited:
            size = explore_size(node)
            if size > largest:
                largest = size
    return largest
```

Q In `size += explore_size(neighbor)` , doesn't neighbor only contribute 1?

No. `explore_size(neighbor)` returns "starting from neighbor, the size of the **entire unvisited region** reachable from there" — which could be far more than 1. It's exactly like Tree's `count_nodes` : `1 + count_nodes(root.left) + count_nodes(root.right)` — `explore_size(neighbor)` plays the exact role of "how many total nodes does this one branch have," except Graph's neighbor count isn't fixed, so it uses a `for` loop to accumulate rather than hardcoding left/right.

11 Grid Graph — 2D Grids

A 2D grid is also a graph: each cell is a node, and its neighbors are the four **up/down/left/right** directions (no diagonals).



Green island (size 4) · orange island (size 1) · red island (size 1) → count_islands = 3

count_islands — DFS + grid islands

```python
def count_islands(grid):
    visited = set()
    island_count = 0
    rows, cols = len(grid), len(grid[0])

    def explore(row, col):
        if row < 0 or row >= rows or col < 0 or col >= cols:
            return                      # out of bounds, stop
        if grid[row][col] == "W":
            return                      # water, stop
        if (row, col) in visited:
            return                      # already visited, stop
        visited.add((row, col))          # tuple as a "coordinate" node
        explore(row + 1, col)            # four directions
        explore(row - 1, col)
        explore(row, col + 1)
        explore(row, col - 1)

    for row in range(rows):
        for col in range(cols):
            if grid[row][col] == "L" and (row, col) not in visited:
                island_count += 1        # found a new island
                explore(row, col)         # mark the whole island (don't collect a return value)
    return island_count
```

Watch the order of the three checks: bounds first, then `grid[row][col]`, then `if...return` to bail early. Checking bounds first matters — indexing `grid[row][col]` with an out-of-range coordinate would crash before you ever get to check whether it's water.

- **minimum_island — Find the smallest island**

explore_size returns a size, not just "explored" — grid size

```
def minimum_island(grid):
    visited = set()
    rows, cols = len(grid), len(grid[0])
    smallest = None

    def explore_size(row, col):
        if row < 0 or row >= rows or col < 0 or col >= cols:
            return 0
        if grid[row][col] == "W":
            return 0
        if (row, col) in visited:
            return 0
        visited.add((row, col))
        return (1
                + explore_size(row + 1, col)
                + explore_size(row - 1, col)
                + explore_size(row, col + 1)
                + explore_size(row, col - 1))

    for row in range(rows):
        for col in range(cols):
            if grid[row][col] == "L" and (row, col) not in visited:
                size = explore_size(row, col)   # this island's total size
                if smallest is None or size < smallest:
                    smallest = size             # only update here

    return 0 if smallest is None else smallest
```

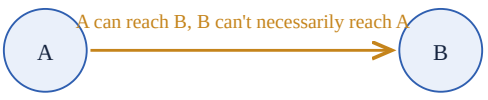
Q I put the "update smallest" logic inside the outer loop, executing for every cell — why is that wrong?

The broken version looks like: `if grid[row][col]=="L" and (row,col) not in visited: size = explore(...)` as one block, with `if smallest is None or size < smallest: smallest = size` as a **separately indented, independent** block. That causes even water cells to try comparing whatever `size` was last computed — `size` might not even have been assigned yet. The fix is to nest "update smallest" **inside** the "found a new island" `if` block — only after genuinely discovering a new island and computing its fresh `size` should you compare and update. The problem isn't whether to pre-initialize `size = None` — it's **indentation placement**.

`Bounds check` `visited stores (row, col) tuples` `No islands → return 0` `Entire grid is land → one big island`

12 Directed Graph

An undirected edge A--B works both ways. A directed edge A→B can **only** go from A to B — it's not guaranteed you can get back. When building the graph, only add one direction.



Only one append: graph[A].append(B)

build_directed_graph — add only one direction — directed

```
def build_directed_graph(edges):
    graph = {}
    for node_a, node_b in edges:
        if node_a not in graph:
            graph[node_a] = []
        if node_b not in graph:
            graph[node_b] = []          # even with no outgoing edges, still create an empty list — avoids
KeyError later
        graph[node_a].append(node_b)     # add this one direction only
    return graph
```

has_directed_path — BFS along the direction of the edges directed

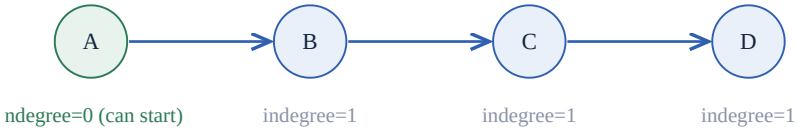
```python
def has_directed_path(graph, start, target):
    if start not in graph or target not in graph:
        return False
    visited = {start}
    queue = [start]
    while len(queue) > 0:
        node = queue.pop(0)
        if node == target:
            return True
        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)
    return False
```

Q I wrote `if start or target not in graph:` — why is the logic wrong?

Python reads this as `if (start) or (target not in graph)` — meaning "true as long as start isn't an empty string," which is completely different from what you meant ("either start or target is missing from the graph"). The correct version needs `not in graph` applied to **each** variable separately: `if start not in graph or target not in graph:`. This is a very easy Python operator-precedence trap to fall into — remember to complete every condition fully whenever you write multi-part conditionals.

13 Topological Sort

Used on a Directed Acyclic Graph (DAG) to produce an order satisfying all dependency relationships. Core concept: **indegree = how many prerequisites are still unfinished**.



A→B→C→D: A has no prerequisite and can go first; the rest must wait for the one before them

topological_sort — Kahn's algorithm kahn

```python
def topological_sort(graph):
    indegree = {}
    for node in graph:
        indegree[node] = 0
    for node in graph:
        for neighbor in graph[node]:
            indegree[neighbor] += 1          # every edge gives its endpoint one more "prerequisite"

    queue = [node for node in indegree if indegree[node] == 0]   # start with everything ready to go
    result = []
    while len(queue) > 0:
        node = queue.pop(0)
        result.append(node)
        for neighbor in graph[node]:
            indegree[neighbor] -= 1           # one of its prerequisites just finished
            if indegree[neighbor] == 0:
                queue.append(neighbor)         # no prerequisites left, can start now
    return result
```

Q For A→B→C→D, how does the algorithm actually run step by step?

Initial state: `indegree = {A:0, B:1, C:1, D:1}`, `queue = [A]`. Pop A → result [A], B's indegree drops 1→0, B gets queued. Pop B → result [A,B], C's indegree drops 1→0, C gets queued. Pop C → result [A,B,C], D's indegree drops 1→0, D gets queued. Pop D → result `[A, B, C, D]`, queue empty, done.

14 Bug Comparison Table

WRONG	WHY IT'S WRONG	CORRECT
Forgetting `visited = set()`	A cycle causes an infinite loop	Always initialize visited before traversing
`if start or target not in graph`	Parsed as `if start` (basically always true)	`not in graph` on each variable separately
`graph[x].append(y)` only (undirected)	Missing the reverse edge	Also add `graph[y].append(x)`
Missing bounds check on row/col	Indexing out of range crashes	Check bounds before indexing grid
Grid visited stores plain values	Grid nodes are coordinates	`visited.add((row, col))`
minimum_island update logic mis-indented	Water cells participate in the comparison too	Nest the update inside the "new island found" branch
DFS used for shortest_path	First path found isn't guaranteed shortest	Use BFS, carry distance in the queue

15 Exercises

Grouped by what's essential vs. what's worth practicing further.

◆ MUST MASTER (BASICS)

1. `Easy` Implement `has_path` on a sample graph and verify it against reachable/unreachable targets.
2. `Easy` Implement `count_components` and confirm it against the 3-component example in § 07.

◆ WORTH PRACTICING (APPLICATIONS)

1. `Medium` Implement `shortest_path` and confirm it returns fewer steps than a DFS-based version on a graph with multiple routes.
2. `Medium` Implement `largest_component`, and explain out loud why `size += explore_size(neighbor)` is correct.

◆ GRID & DIRECTED (MEDIUM)

1. `Medium` Implement `count_islands` and `minimum_island` on the sample grid in § 11.
2. `Medium` Implement `has_directed_path`, and confirm reachability is **not** symmetric on a directed graph.
3. `Medium` Implement `topological_sort` on the A→B→C→D example, and hand-trace the indegree table at each step.